

DAY 1 – JavaScript Fundamentals

◆ What is JavaScript?

👉 JavaScript is a **programming language** used to:

- Make websites interactive
- Build apps (React, Node.js)
- Handle logic in the browser

💡 Example: Without JS → static page

With JS → button click, animations, forms

◆ Variables (MOST IMPORTANT)

Variables store data.

✅ Types:

JavaScript

```
var a = 10;
```

```
let b = 20;
```

```
const c = 30;
```

Show more lines

✅ Difference:

Keyword	Scope	Can Reassign	Can Redeclare
var	function	✅	✅
let	block	✅	❌
const	block	❌	❌

🔥 Best Practice:

- Use `const` by default
 - Use `let` if value changes
 - Avoid `var`
-

◆ Data Types

✓ Primitive:

JavaScript

string → "Hello"

number → 10

boolean → true/false

null

undefined

Show more lines

✓ Non-Primitive:

JavaScript

object → {name: "Sameer"}

array → [1, 2, 3]

Show more lines

◆ Example:

JavaScript

```
let name = "Sameer";
```

```
let age = 25;
```

```
let isDev = true;
```

```
console.log(name);
```

```
``
```

Show more lines

◆ Conditionals

JavaScript
let age = 18;

```
if (age >= 18) {  
  console.log("Adult");  
} else {  
  console.log("Minor");  
}
```

Show more lines

◆ Functions

Normal Function:

JavaScript
function greet(name) {
 return "Hello " + name;
}
..

Show more lines

Arrow Function:

JavaScript
const greet = (name) => {
 return "Hello " + name;
};

Show more lines



PRACTICE (DO THIS TODAY)



Questions:

1. Create a variable and store your name
2. Write a function that adds two numbers
3. Check if a number is even or odd

4. Print numbers from 1–10
 5. Create an object for a student
-

MINI CHALLENGE

JavaScript

// Input: 5

// Output: "Odd"

```
function check(num) {  
  // write logic  
}
```

Show more lines

NEXT STEP (IMPORTANT)

Tomorrow we will cover: 👉 Arrays + Objects deeply (VERY IMPORTANT FOR REACT)

DAY 2 — Arrays & Objects (MOST IMPORTANT FOR REACT)

👉 This is **one of the most important topics** in JavaScript.

React is basically **manipulating arrays + objects + UI**

◆ PART 1: Arrays

✅ What is an Array?

👉 A collection of values stored in a single variable

JavaScript

```
let numbers = [1, 2, 3, 4, 5];
```

Show more lines

📌 Java comparison:

- Java → `int[] arr = new int[5];`
 - JS → dynamic, no fixed size
-

✅ Access Values

JavaScript

```
let fruits = ["apple", "banana", "mango"];
```

```
console.log(fruits[0]); // apple
```

```
console.log(fruits[2]); // mango
```

Show more lines

✅ Common Array Methods (VERY IMPORTANT)

1. `push()` → add element at end

JavaScript

```
let arr = [1, 2];
```

```
arr.push(3); // [1, 2, 3]
```

Show more lines

2. `pop()` → remove last element

JavaScript

```
arr.pop(); // [1, 2]
```

Show more lines

3. `shift()` → remove first

JavaScript

```
arr.shift();
```

Show more lines

4. **unshift()** → add at beginning

JavaScript

```
arr.unshift(10);
```

Show more lines

MOST IMPORTANT (Used in React)

map() → transform array

JavaScript

```
let nums = [1, 2, 3];
```

```
let doubled = nums.map((n) => n * 2);
```

```
console.log(doubled); // [2, 4, 6]
```

Show more lines

 Real use case:  Convert data → UI

filter() → condition check

JavaScript

```
let nums = [1, 2, 3, 4];
```

```
let even = nums.filter((n) => n % 2 === 0);
```

```
console.log(even); // [2, 4]
```

Show more lines

reduce() → accumulate value

JavaScript

```
let nums = [1, 2, 3];
```

```
let sum = nums.reduce((acc, curr) => acc + curr, 0);
```

```
console.log(sum); // 6
```

Show more lines

◆ PART 2: Objects

✓ What is an Object?

👉 Collection of key-value pairs

JavaScript

```
let person = {  
  name: "Sameer",  
  age: 25,  
  isDev: true  
};
```

Show more lines

✓ Access Object Data

JavaScript

```
console.log(person.name); // Sameer  
console.log(person["age"]); // 25
```

Show more lines

✓ Update Object

JavaScript

```
person.age = 26;
```

Show more lines

✓ Add new property

JavaScript

```
person.city = "Kolkata";
```

Show more lines

✓ Nested Object

JavaScript

```
let user = {  
  name: "Sameer",  
  address: {  
    city: "Kolkata",  
    pincode: 700001  
  }  
};
```

```
console.log(user.address.city);
```

Show more lines

🔥 Arrays + Objects Together (REAL WORLD)

👉 This is EXACTLY how API data comes

JavaScript

```
let users = [  
  { name: "Sameer", age: 25 },  
  { name: "Rahul", age: 30 }  
];
```

Show more lines

✓ Loop through objects

JavaScript

```
users.map((user) => {  
  console.log(user.name);  
});
```

Show more lines



React Insight (IMPORTANT)

👉 In React, you will:

- Store data in arrays
- Render UI using `map()`

Example:

JavaScript

```
users.map((user) => <p>{user.name}</p>);
```

Show more lines

✓ PRACTICE (DO THIS TODAY)

🧠 Questions:

1. Array Practice:

- Create array of 5 numbers
 - Add 1 element
 - Remove last element
-

2. `map()`:

JavaScript

```
// Input: [1, 2, 3]
```

```
// Output: [1, 4, 9]
```

Show more lines

3. filter():

JavaScript

```
// Input: [10, 15, 20, 25]
```

```
// Output: [15, 25]
```

Show more lines

4. Object:

Create:

JavaScript

```
student = {
```

```
  name,
```

```
  age,
```

```
  marks
```

```
}
```

Show more lines

5. Complex:

JavaScript

```
let products = [
```

```
{ name: "Laptop", price: 50000 },
```

```
{ name: "Phone", price: 20000 }
```

```
];
```

```
// Print only products > 30000
```

Show more lines

MINI CHALLENGE

JavaScript

```
let users = [  
  { name: "Sameer", age: 25 },  
  { name: "Rahul", age: 17 },  
  { name: "Ankit", age: 30 }  
];
```

// Output: ["Sameer", "Ankit"]

Show more lines

👉 Use: filter + map

DAY 3 — DOM Manipulation + Events

👉 This is where JavaScript becomes **interactive**

◆ PART 1: What is DOM?

DOM = Document Object Model

👉 When a webpage loads, the browser converts HTML into a tree structure.

Example HTML:

HTML

```
<h1 id="title">Hello</h1>  
<button>Click Me</button>
```

Show more lines

👉 JS sees it like:

```
None
Document
└─ h1
└─ button
```

👉 You can **read, change, add, delete** elements using JS

◆ PART 2: Selecting Elements

✓ 1. getElementById

JavaScript

```
let title = document.getElementById("title");
console.log(title);
``
```

Show more lines

✓ 2. querySelector (MOST IMPORTANT)

JavaScript

```
let title = document.querySelector("#title");
```

Show more lines

👉 Works like CSS:

JavaScript

```
document.querySelector(".className")
```

```
document.querySelector("h1")
```

Show more lines

✓ 3. `querySelectorAll`

JavaScript

```
let allButtons = document.querySelectorAll("button");
```

Show more lines

◆ PART 3: Changing Content

✓ Change Text

JavaScript

```
title.innerText = "Hello Sameer!";
```

Show more lines

✓ Change HTML

JavaScript

```
title.innerHTML = "<span>New Text</span>";
```

Show more lines

✓ Change Style

JavaScript

```
title.style.color = "red";
```

```
title.style.fontSize = "30px";
```

Show more lines

◆ PART 4: Events (INTERACTION 🔥)

👉 Events = user actions
(click, type, hover, etc.)

✓ Click Event

JavaScript

```
let btn = document.querySelector("button");
```

```
btn.addEventListener("click", () => {  
  console.log("Button clicked!");  
});
```

Show more lines

✓ Example: Change Text on Click

JavaScript

```
let btn = document.querySelector("button");  
let title = document.querySelector("h1");
```

```
btn.addEventListener("click", () => {  
  title.innerText = "You clicked!";  
});
```

Show more lines

REAL WORLD MINI APP

HTML

```
<h1 id="counter">0</h1>  
<button id="increase">Increase</button>
```

Show more lines

JavaScript

```
let count = 0;
```

```
let counter = document.getElementById("counter");  
let btn = document.getElementById("increase");
```

```
btn.addEventListener("click", () => {  
  count++;  
  counter.innerText = count;  
});
```

Show more lines

💡 This is your **first interactive app** ✅

👉 Same concept → React later

◆ PART 5: Input Handling

HTML

```
<input type="text" id="name"/>  
<button>Submit</button>
```

Show more lines

JavaScript

```
let input = document.querySelector("#name");  
let btn = document.querySelector("button");
```

```
btn.addEventListener("click", () => {  
  console.log(input.value);  
});
```

Show more lines

🔥 KEY CONCEPT (VERY IMPORTANT)

👉 JS does:

1. Select element
2. Listen event

3. Update UI

💡 This is EXACTLY what React does behind the scenes

PRACTICE (DO THIS TODAY)

Q1:

- Create a button
 - On click → print "Hello"
-

Q2:

- Change background color when button clicked
-

Q3:

- Input box → print entered value
-

Q4:

Create counter app:

- Increase
 - Decrease
-

MINI CHALLENGE

👉 Build this logic:

HTML

```
<h1>0</h1>
```

```
<button>+</button>
```

```
<button>-</button>
```

Show more lines

✓ Output:

- → increase
 - → decrease
-



INTERVIEW TIP

👉 Difference:

- `innerText` → only text
- `innerHTML` → allows tags



DAY 4 — Scope, Hoisting & Execution Context

◆ PART 1: Scope

👉 Scope decides **where a variable is accessible**

✓ Types of Scope

1. Global Scope

JavaScript

```
let name = "Sameer";
```

```
function print() {  
  console.log(name);  
}
```

```
print(); // ✅ Accessible everywhere
```

Show more lines

2. Function Scope

JavaScript

```
function test() {  
  let x = 10;  
  console.log(x);  
}
```

```
test();
```

```
// console.log(x); ❌ Error (outside function)
```

Show more lines

3. Block Scope (VERY IMPORTANT)

JavaScript

```
{  
  let a = 10;  
  const b = 20;  
}
```

```
// console.log(a); ❌ Error
```

```
// console.log(b); ❌ Error
```

Show more lines

👉 But **var** behaves differently:

JavaScript

```
{  
var x = 100;  
}
```

console.log(x); //  Works

Show more lines

 **Key Point:**

- `let` & `const` → block scope 
- `var` → NOT block scoped 

PART 2: Hoisting

 Hoisting = JS moves **declarations to top**

Example 1

JavaScript

```
console.log(a);
```

```
var a = 10;
```

Show more lines

 Output:

None

undefined

 Internally:

JavaScript

```
var a;
```

```
console.log(a);
```

```
a = 10;
```

Show more lines

✓ Example 2 (IMPORTANT)

JavaScript

```
console.log(b);
```

```
let b = 20;
```

Show more lines

👉 Output:

None

ReferenceError ✖

💡 Reason:

- `let` & `const` are hoisted BUT **not initialized**
-

✓ Functions Hoisting

Works:

JavaScript

```
hello();
```

```
function hello() {  
  console.log("Hi");  
}
```

Show more lines

Does NOT work:

JavaScript

```
hello();
```

```
const hello = () => {  
  console.log("Hi");  
};
```

Show more lines

PART 3: Execution Context (CORE CONCEPT)

👉 JS runs code in **two phases**

✓ 1. Memory Phase (Creation Phase)

- Variables set as **undefined**
 - Functions stored completely
-

✓ 2. Execution Phase

- Code runs line by line
 - Values are assigned
-

✓ Example

JavaScript
var x = 10;

```
function test() {  
  var y = 20;  
}
```

test();

Show more lines

👉 What happens:

Step 1: Memory

```
None
x = undefined
test = function stored
```

Step 2: Execution

```
None
x = 10
test() → creates new execution context
```

Execution Context Visualization

```
None
Global Execution Context (GEC)
|
|-- test() Execution Context
```

👉 Whenever a function runs → new context created

CALL STACK (VERY IMPORTANT)

👉 JS uses stack:

```
JavaScript
function a() {
  b();
}

function b() {
  console.log("Inside b");
}
```

a();

Show more lines

Stack:

```
None
Global
a()
b()
```

👉 After execution:

```
None
Global
```



REAL INTERVIEW TRICK

```
JavaScript
var x = 1;

function test() {
  console.log(x);
  var x = 2;
}
```

```
test();
```

Show more lines

👉 Output:

None

undefined

👉 Why?

Internally:

JavaScript

```
function test() {
```

```
  var x;
```

```
  console.log(x);
```

```
  x = 2;
```

```
}
```

Show more lines



PRACTICE (VERY IMPORTANT)

🧠 Q1:

JavaScript

```
console.log(a);
```

```
let a = 5;
```

Show more lines

🧠 Q2:

JavaScript

```
var x = 10;
```

```
function test() {
```

```
  console.log(x);
```



```
}
```

```
test();
```

Show more lines

Q3:

JavaScript

```
function test() {  
  console.log(a);  
  var a = 100;  
}
```

```
test();
```

Show more lines

Q4:

JavaScript

```
let a = 10;
```

```
{  
  let a = 20;  
  console.log(a);  
}
```

```
console.log(a);
```

Show more lines

MINI CHALLENGE

JavaScript

```
var a = 5;
```

```
function outer() {
```

```
var a = 10;
```

```
function inner() {  
  console.log(a);  
}
```

```
inner();  
}
```

```
outer();
```

Show more lines

👉 Predict output ☒

KEY TAKEAWAY (MEMORY TRICK)

👉 Remember:

- Scope → "Where can I access?"
- Hoisting → "Declaration moves up"
- Execution Context → "How JS runs code"

DAY 5 — Closures (VERY IMPORTANT)

👉 If you understand closures deeply, you'll:

- Crack JS interviews ☒
- Understand React hooks ☒
- Write advanced logic ☒

◆ PART 1: What is a Closure?

👉 Closure = Function + its surrounding variables (scope)

👉 A function “remembers” variables from where it was created.

✅ Basic Example

JavaScript

```
function outer() {  
  let a = 10;
```

```
  function inner() {  
    console.log(a);  
  }  
}
```

```
  return inner;  
}
```

```
let result = outer();  
result(); // ✅ Output: 10  
Show more lines
```

🔥 What Happened?

Even after `outer()` finished execution,

👉 `inner()` STILL remembers `a`

💡 This is a **closure**

◆ PART 2: Visual Understanding

```
None
outer() runs → creates a = 10
  ↓
returns inner()
  ↓
inner() still remembers a = 10 ✓
```

◆ PART 3: Real Life Analogy

👉 Think like this:

- Outer function = house 🏠
- Inner function = person 👤
- Variable = memory 🧠

Even if house is gone → person remembers memory

◆ PART 4: Practical Example

✓ Counter using Closure

```
JavaScript
function counter() {
  let count = 0;

  return function () {
    count++;
    console.log(count);
  };
};
```

```
}
```

```
let c = counter();
```

```
c(); // 1
```

```
c(); // 2
```

```
c(); // 3
```

Show more lines

👉 Why it works?

- `count` is inside outer function
 - Inner function keeps access to it ✅
-

HUGE INTERVIEW QUESTION

✅ Data Encapsulation (Private Variables)

JavaScript

```
function bankAccount() {
```

```
  let balance = 1000;
```

```
  return {
```

```
    deposit: function(amount) {
```

```
      balance += amount;
```

```
      console.log(balance);
```

```
    },
```

```
    withdraw: function(amount) {
```

```
      balance -= amount;
```

```
      console.log(balance);
```

```
    }
```

```
  };
```

```
}
```

```
let account = bankAccount();
```

```
account.deposit(500); // 1500
```

```
account.withdraw(200); // 1300
```

Show more lines

👉 You **cannot access balance directly** ❌

👉 Only via functions ✅

💡 This is used in real applications

◆ PART 5: Common Mistake ⚠️

❌ Problem with **var**

JavaScript

```
for (var i = 1; i <= 3; i++) {  
  setTimeout(() => {  
    console.log(i);  
  }, 1000);  
}
```

Show more lines

👉 Output:

None

4

4

4

✅ Fix using **let**

JavaScript

```
for (let i = 1; i <= 3; i++) {  
  setTimeout(() => {  
    console.log(i);  
  }, 1000);  
}
```

Show more lines

👉 Output:

None

1

2

3

◆ PART 6: Why Closures Matter for React

👉 React uses closures in:

- useState
- event handlers
- hooks logic

Example:

JavaScript

```
function App() {  
  let count = 0;
```

```
  function handleClick() {  
    count++;  
    console.log(count);  
  }  
}
```

Show more lines

👉 `handleClick` remembers `count`

KEY TAKEAWAY

👉 Closure =

- ✅ Function
- ✅ + Access to outer scope
- ✅ Even after execution

✅ PRACTICE (VERY IMPORTANT)

🧠 Q1:

JavaScript

```
function outer() {  
  let x = 5;
```

```
  return function inner() {  
    console.log(x);  
  };  
}
```

```
outer()();
```

Show more lines

👉 Output?

🧠 Q2:

JavaScript

```
function test() {  
  let count = 0;
```



```
return function () {  
  count += 2;  
  console.log(count);  
};  
}
```

```
let t = test();  
t();  
t();  
``
```

Show more lines

Q3:

Create your own counter function using closure 

MINI CHALLENGE

 Create this:

```
JavaScript  
function secret() {  
  let data = "Hidden";  
  
  return function () {  
    return data;  
  };  
}
```

Show more lines

 Expected:

None

`secret()()` → "Hidden"

INTERVIEW GOLD LINE

👉 If asked:

"What is closure?"

Say:

👉 *"Closure is a function that remembers its lexical scope even after the outer function has finished execution."*

DAY 6 — Promises & Async/Await (VERY IMPORTANT)

👉 This topic is **CRITICAL** for:

- API calls 
- React apps 
- Backend integration 

◆ PART 1: The Problem (Why Async?)

👉 JavaScript is **single-threaded**

→ It executes one line at a time

But what about:

- API calls (takes time)

- File reading
- Database calls

👉 We don't want to block execution ❌

✅ Example Problem

JavaScript

```
console.log("Start");
```

```
setTimeout(() => {  
  console.log("Data loaded");  
}, 2000);
```

```
console.log("End");
```

Show more lines

👉 Output:

None

Start

End

Data loaded

💡 JS does NOT wait → runs asynchronously

◆ PART 2: What is a Promise?

👉 Promise = a value that will be available **in the future**

✅ Promise States

1. Pending ⌚
 2. Resolved ✅
 3. Rejected ❌
-

✅ Create a Promise

JavaScript

```
let promise = new Promise((resolve, reject) => {  
  let success = true;
```

```
  if (success) {  
    resolve("Success ✅");  
  } else {  
    reject("Failed ❌");  
  }  
});
```

Show more lines

✅ Consume Promise (then / catch)

JavaScript

```
promise  
  .then((res) => console.log(res))  
  .catch((err) => console.log(err));
```

Show more lines

🔥 REAL EXAMPLE (API Simulation)

JavaScript

```
function fetchData() {  
  return new Promise((resolve, reject) => {  
    setTimeout(() => {  
      resolve("Data received ✅");
```

```
}, 2000);  
});  
}
```

```
fetchData().then((data) => console.log(data));
```

Show more lines

◆ PART 3: Callback Hell 🤯

👉 Before promises:

```
JavaScript  
setTimeout(() => {  
  console.log("Step 1");
```

```
  setTimeout(() => {  
    console.log("Step 2");
```

```
    setTimeout(() => {  
      console.log("Step 3");  
    }, 1000);
```

```
  }, 1000);
```

```
}, 1000);
```

Show more lines

👉 Hard to read & maintain ❌

◆ PART 4: Promises solve this ✅

```
JavaScript  
fetchData()
```

```
.then(() => fetchData())  
.then(() => fetchData())  
.catch((err) => console.log(err));
```

Show more lines

◆ PART 5: Async / Await (MODERN 🔥)

👉 Makes async code look like **normal code**

✅ Example

JavaScript

```
function fetchData() {  
  return new Promise((resolve) => {  
    setTimeout(() => resolve("Data ✅"), 2000);  
  });  
}
```

Show more lines

✅ Using Async/Await

JavaScript

```
async function getData() {  
  let result = await fetchData();  
  console.log(result);  
}
```

```
getData();  
..
```

Show more lines

💡 Behind the scenes:

- `await` pauses execution (only inside async function)
-

◆ PART 6: Error Handling

```
JavaScript
async function getData() {
  try {
    let result = await fetchData();
    console.log(result);
  } catch (error) {
    console.log(error);
  }
}
```

Show more lines



REAL API CALL (VERY IMPORTANT)

```
JavaScript
async function getUsers() {
  let res = await fetch("https://jsonplaceholder.typicode.com/users");
  let data = await res.json();

  console.log(data);
}
```

getUsers();

Show more lines

👉 This is how React apps fetch data ✅



KEY DIFFERENCE

Feature	Promise (.then)	Async/Await
Syntax	Chain	Clean
Readability	Medium	High ✓
Error handling	.catch	try/catch ✓



MEMORY TRICK

👉 Promise = "I will give result later"

👉 await = "Wait until result comes"



PRACTICE (VERY IMPORTANT)

🧠 Q1:

JavaScript

```
let p = new Promise((resolve) => {  
  resolve("Hello");  
});
```

```
p.then((res) => console.log(res));
```

Show more lines

👉 Output?

🧠 Q2:

Convert this to async/await:

JavaScript

```
fetchData().then((data) => console.log(data));
```

Show more lines

Q3:

Create a promise:

- resolves after 2 seconds
 - prints "Done"
-

Q4:

Handle error using try/catch

MINI CHALLENGE

JavaScript

```
async function test() {  
  return "Hello";  
}
```

```
test().then((res) => console.log(res));
```

Show more lines

👉 Output?

INTERVIEW GOLD

👉 Question: Difference between async/await & promises?

✓ Answer:

- Async/await is syntactic sugar over promises
- Makes code cleaner and easier to read

DAY 7 — Event Loop + Call Stack + Web APIs

👉 This is what makes JavaScript **asynchronous magic work**

BIG PICTURE (VERY IMPORTANT)

JavaScript uses:

1. ✓ Call Stack
 2. ✓ Web APIs
 3. ✓ Callback Queue
 4. ✓ Event Loop
-

◆ PART 1: Call Stack

👉 Call Stack = where JS executes code

✓ **Example:**

```
JavaScript
function a() {
  b();
}
```

```
function b() {  
  console.log("Inside b");  
}
```

a();

Show more lines

✓ Execution Flow:

Stack:

```
None  
Global()  
a()  
b()
```

👉 Output:

```
None  
Inside b
```

👉 Then stack clears:

```
None  
Global()
```

◆ PART 2: Blocking vs Non-Blocking

✗ Blocking Code:

JavaScript

```
console.log("Start");
```

```
while (true) {} // blocks everything
```

```
console.log("End"); // never runs
```

Show more lines

✅ Non-Blocking (Async):

JavaScript

```
setTimeout(() => {  
  console.log("Async Task");  
}, 2000);
```

Show more lines

👉 Does NOT block execution ✅

◆ PART 3: Web APIs

👉 Browser provides:

- setTimeout
- DOM
- fetch
- event listeners

👉 These run **outside JS engine**

🔥 Example:

JavaScript

```
console.log("Start");
```

```
setTimeout(() => {  
  console.log("Timer Done");  
}, 2000);
```

```
console.log("End");
```

Show more lines

✓ Flow:

1. **Start** → printed
2. `setTimeout` → goes to **Web API**
3. **End** → printed
4. After 2s → callback goes to queue
5. Event loop pushes it to stack

👉 Output:

None

Start

End

Timer Done

◆ PART 4: Callback Queue

👉 Stores completed async callbacks

Example:

None

`setTimeout` → goes here after completion

`click events` → queued here

PART 5: Event Loop (Brain of JS)

👉 Event Loop keeps checking:

None

IF Call Stack is empty ☒

THEN take task from queue → push to stack

Visualization

None

Call Stack ← Event Loop ← Callback Queue

↑

Web APIs

SUPER IMPORTANT EXAMPLE

JavaScript

```
console.log("Start");
```

```
setTimeout(() => {  
  console.log("Timeout");  
}, 0);
```

```
console.log("End");
```

Show more lines

☒ **Output:**

None

Start

End

Timeout

👉 Why NOT immediate?

💡 Because:

- Even 0ms → goes to Web API → queue → waits

♦ PART 6: Microtask Queue (ADVANCED 🔥)

👉 Higher priority than callback queue

👉 Used by:

- Promises
- async/await

✅ Example:

JavaScript

```
console.log("Start");
```

```
Promise.resolve().then(() => {  
  console.log("Promise");  
});
```

```
console.log("End");
```

Show more lines

✓ **Output:**

None

Start

End

Promise

FINAL PRIORITY ORDER

1. Call Stack ✓
 2. Microtask Queue (Promises) 🔥
 3. Callback Queue (setTimeout)
-

CRAZY INTERVIEW QUESTION

JavaScript

```
console.log("Start");
```

```
setTimeout(() => console.log("Timeout"), 0);
```

```
Promise.resolve().then(() => console.log("Promise"));
```

```
console.log("End");
```

Show more lines

✓ **Output:**

None

Start

End

Promise

Timeout

✓ WHY?

1. Sync runs first ✓
 2. Promise → microtask (HIGH PRIORITY)
 3. Timeout → callback queue (LOW PRIORITY)
-

◆ PART 7: Real Life Analogy

👉 Think:

- Call Stack = Chef 🍳 cooking current dish
 - Web APIs = Kitchen team 👥
 - Queue = waiting orders 📋
 - Event Loop = Manager 👤 assigning work
-

✓ PRACTICE (VERY IMPORTANT)

🧠 Q1:

JavaScript

```
console.log("A");
```

```
setTimeout(() => console.log("B"), 0);
```

```
console.log("C");
```

Show more lines

👉 Output?

Q2:

JavaScript

```
console.log("A");
```

```
Promise.resolve().then(() => console.log("B"));
```

```
console.log("C");
```

Show more lines

Q3:

JavaScript

```
setTimeout(() => console.log("Timeout"), 0);
```

```
Promise.resolve().then(() => console.log("Promise"));
```

```
..
```

Show more lines

MINI CHALLENGE

JavaScript

```
console.log("1");
```

```
setTimeout(() => console.log("2"), 0);
```

```
Promise.resolve().then(() => console.log("3"));
```

```
console.log("4");
```

Show more lines

👉 Predict output ✓

🔥 KEY TAKEAWAY

👉 JavaScript is:

- Single-threaded ✓
- Non-blocking ✓
- Event-driven ✓

👉 Event Loop = magic behind async

🚀 DAY 8 — **this** Keyword + call, apply, bind

👉 This is used **everywhere**:

- Objects
 - Functions
 - React (especially older class components)
 - Event handlers
-

🔥 PART 1: What is **this**?

👉 **this** refers to **the object that is calling the function**

💡 Simple rule: 👉 “Who is calling the function?” = **this**

◆ PART 2: **this** in Different Cases

✓ 1. Global Context

JavaScript

```
console.log(this);
```

Show more lines

👉 In browser:

None

window object

✓ 2. Inside Object

JavaScript

```
let person = {  
  name: "Sameer",  
  greet: function () {  
    console.log(this.name);  
  }  
};
```

```
person.greet();
```

Show more lines

👉 Output:

None

Sameer

👉 `this = person`

✅ 3. Inside Normal Function

JavaScript

```
function test() {  
  console.log(this);  
}
```

test();

Show more lines

👉 Output:

- Browser → `window`
 - Strict mode → `undefined`
-

✅ 4. Arrow Function (IMPORTANT 🔥)

👉 Arrow functions **do NOT** have their own `this`

JavaScript

```
let obj = {  
  name: "Sameer",  
  greet: () => {  
    console.log(this.name);  
  }  
};
```

obj.greet();

Show more lines

👉 Output:

None

undefined ❌

👉 Why?

- Arrow function takes `this` from **parent scope**

◆ PART 3: Fix Arrow Function Issue

JavaScript

```
let obj = {  
  name: "Sameer",  
  greet() {  
    console.log(this.name);  
  }  
};
```

Show more lines

👉 Output:

None

Sameer ✅

🔥 PART 4: `call()`, `apply()`, `bind()`

👉 These control **what** `this` should be

✅ 1. `call()`

👉 Calls function immediately

JavaScript

```
function greet(city) {  
  console.log(this.name, city);  
}
```

```
let user = { name: "Sameer" };
```

```
greet.call(user, "Kolkata");
```

Show more lines

👉 Output:

None

Sameer Kolkata

✓ 2. apply()

👉 Same as call, but arguments in array

JavaScript

```
greet.apply(user, ["Delhi"]);
```

Show more lines

✓ 3. bind() (VERY IMPORTANT)

👉 Returns a new function

JavaScript

```
let newFunc = greet.bind(user, "Mumbai");
```

```
newFunc();
```

Show more lines

👉 Output:

None

Sameer Mumbai



DIFFERENCE

Method	Execution	Arguments
call	Immediate	Comma separated
apply	Immediate	Array
bind	Later	Comma separated

◆ PART 5: Real Use Case (IMPORTANT



✓ Event Handler Problem

JavaScript

```
let btn = document.querySelector("button");
```

```
let obj = {  
  name: "Sameer",  
  handleClick: function () {  
    console.log(this.name);  
  }  
};
```

```
btn.addEventListener("click", obj.handleClick);
```

Show more lines

👉 Output:

None

undefined ❌

✅ Fix using bind:

JavaScript

```
btn.addEventListener("click", obj.handleClick.bind(obj));
```

Show more lines

👉 Output:

None

Sameer ✅

🔥 PART 6: Real Interview Trick

JavaScript

```
let obj = {  
  name: "Sameer",  
  greet: function () {  
    function inner() {  
      console.log(this.name);  
    }  
    inner();  
  }  
};
```

```
obj.greet();
```

Show more lines

👉 Output:

None

undefined ❌

✅ Fix:

JavaScript

```
let obj = {  
  name: "Sameer",  
  greet: function () {  
    let inner = () => {  
      console.log(this.name);  
    };  
    inner();  
  }  
};
```

Show more lines

👉 Output:

None

Sameer ✅

🔥 MEMORY TRICK

👉 `this` depends on:

- HOW function is called !
 - NOT where it is written !
-

✅ PRACTICE (VERY IMPORTANT)

Q1:

JavaScript

```
let obj = {  
  name: "A",  
  print: function () {  
    console.log(this.name);  
  }  
};
```

obj.print();

Show more lines

Q2:

JavaScript

```
function test() {  
  console.log(this);  
}
```

test();

Show more lines

Q3:

JavaScript

```
let obj = {  
  name: "Sameer",  
  greet: () => {  
    console.log(this.name);  
  }  
};
```

obj.greet();

Show more lines

Q4:

Use `call()` to print name:

JavaScript

```
function show() {  
  console.log(this.name);  
}
```

```
let user = { name: "Sameer" };
```

Show more lines

MINI CHALLENGE

JavaScript

```
let user1 = { name: "Sameer" };
```

```
let user2 = { name: "Rahul" };
```

```
function greet() {  
  console.log(this.name);  
}
```

```
// Print Sameer
```

```
// Print Rahul
```

Show more lines

👉 Use call/apply/bind ☒

INTERVIEW GOLD

👉 Question: Difference between arrow & normal function?

☒ Answer:

- Arrow → no own `this`
- Normal → has its own `this`

DAY 9 — Prototype & Inheritance

👉 This is how JavaScript **reuses code internally**

PART 1: What is Prototype?

👉 Every JavaScript object has a hidden property:

None

`[[Prototype]]`

- 👉 It is used to: ☒ Inherit properties
- ☒ Share methods
- ☒ Avoid duplication
-

☒ Example

JavaScript

```
let obj = {  
  name: "Sameer"  
};
```

```
console.log(obj.__proto__);
```

Show more lines

👉 `__proto__` points to:

None

`Object.prototype`

◆ PART 2: Why Prototype Exists?

👉 Without prototype:

JavaScript

```
let user1 = {  
  name: "Sameer",  
  greet: function () {  
    console.log("Hello");  
  }  
};
```

```
let user2 = {  
  name: "Rahul",  
  greet: function () {  
    console.log("Hello");  
  }  
};
```

Show more lines

✗ Problem: duplicate code



With Prototype

JavaScript

```
function User(name) {  
  this.name = name;  
}
```

```
User.prototype.greet = function () {
```

```
console.log("Hello " + this.name);  
};
```

```
let u1 = new User("Sameer");  
let u2 = new User("Rahul");
```

```
u1.greet();  
u2.greet();  
Show more lines
```

- ✓ One function shared
 - ✓ Memory efficient
-

PART 3: Prototype Chain

👉 If property not found in object → JS searches in prototype chain

✓ Example

JavaScript

```
let arr = [1, 2, 3];
```

```
console.log(arr.length); // own property  
console.log(arr.push); // from prototype  
Show more lines
```

👉 Chain looks like:

None

```
arr → Array.prototype → Object.prototype → null
```

◆ PART 4: Inheritance

👉 One object can inherit properties from another

✅ Using Object.create()

JavaScript

```
let parent = {  
  greet() {  
    console.log("Hello");  
  }  
};
```

```
let child = Object.create(parent);
```

```
child.greet(); // ✅ Inherited
```

Show more lines

👉 Internally:

None

```
child → parent → Object.prototype
```

🔥 PART 5: Constructor Functions (IMPORTANT)

👉 Before ES6 classes, JS used this

✓ Example

JavaScript

```
function Person(name, age) {  
  this.name = name;  
  this.age = age;  
}
```

```
Person.prototype.sayHi = function () {  
  console.log("Hi " + this.name);  
};
```

```
let p1 = new Person("Sameer", 25);  
p1.sayHi();
```

Show more lines

💡 **new** keyword does:

1. Creates empty object
 2. Binds **this** to that object
 3. Links to prototype
 4. Returns object
-

◆ PART 6: ES6 Classes (Modern Way ✓)

👉 Easier syntax over prototype

✓ Example

JavaScript

```
class Person {  
  constructor(name, age) {  
    this.name = name;
```

```
this.age = age;
}

sayHi() {
  console.log("Hi " + this.name);
}
}

let p1 = new Person("Sameer", 25);
p1.sayHi();
Show more lines
```

💡 Under the hood: 🙌 Still uses **prototype**

PART 7: Inheritance in Classes

```
JavaScript
class Animal {
  speak() {
    console.log("Animal speaks");
  }
}

class Dog extends Animal {
  bark() {
    console.log("Dog barks");
  }
}

let d = new Dog();

d.speak(); // ✅ inherited
d.bark();
Show more lines
```

PART 8: Real Interview Trick

JavaScript

```
function A() {}
```

```
function B() {}
```

```
B.prototype = Object.create(A.prototype);
```

```
let obj = new B();
```

```
console.log(obj instanceof A); // ?
```

Show more lines

👉 Output:

None

true 

👉 Because prototype linked

MEMORY TRICK

👉 Prototype =

 "Backup object"

 Used when property not found

 **PRACTICE (VERY IMPORTANT)**

 Q1:

JavaScript

```
let obj = {};  
console.log(obj.__proto__);
```

Show more lines

 Q2:

JavaScript

```
function Test() {}  
let t = new Test();  
  
console.log(t.__proto__ === Test.prototype);
```

Show more lines

 Q3:

Create constructor:

JavaScript

Car(name)

Show more lines

Add method:

None

`drive()`

 Q4:

Use `Object.create` to inherit



MINI CHALLENGE

JavaScript

```
class Person {  
  constructor(name) {  
    this.name = name;  
  }  
}
```

```
class Student extends Person {  
  constructor(name, grade) {  
    super(name);  
    this.grade = grade;  
  }  
}
```

```
let s = new Student("Sameer", "A");
```

```
console.log(s.name, s.grade);
```

Show more lines

👉 Predict output ☒



INTERVIEW GOLD

👉 Question: Difference between class & prototype?

☒ Answer:

- Class = syntactic sugar
- Prototype = actual implementation



DAY 10 — ES6 Features (Modern JavaScript)

👉 These features make JS:

- Cleaner ✓
 - Shorter ✓
 - More powerful ✓
-



PART 1: Destructuring (VERY IMPORTANT)

👉 Extract values from arrays/objects easily



Array Destructuring

JavaScript

```
let nums = [10, 20, 30];
```

```
let [a, b, c] = nums;
```

```
console.log(a); // 10
```

```
console.log(b); // 20
```

Show more lines



Skip Values

JavaScript

```
let [a, , c] = [1, 2, 3];
```

```
console.log(a, c); // 1 3
```

Show more lines

✓ Object Destructuring (MOST USED IN REACT)

JavaScript

```
let person = {  
  name: "Sameer",  
  age: 25  
};
```

```
let { name, age } = person;
```

```
console.log(name); // Sameer
```

Show more lines

✓ Rename Variables

JavaScript

```
let { name: userName } = person;
```

```
console.log(userName);
```

Show more lines

✓ Function Parameter Destructuring (🔥 React)

JavaScript

```
function greet({ name }) {  
  console.log(name);  
}
```

Show more lines

🔥 PART 2: Spread Operator (...)

👉 Expands values

✅ Arrays

JavaScript

```
let a = [1, 2];
```

```
let b = [3, 4];
```

```
let result = [...a, ...b];
```

```
console.log(result); // [1,2,3,4]
```

Show more lines

✅ Copy Array

JavaScript

```
let copy = [...a];
```

Show more lines

✅ Objects

JavaScript

```
let user = { name: "Sameer" };
```

```
let updatedUser = { ...user, age: 25 };
```

Show more lines

💡 React Insight: 👉 Used for updating state without mutation ✅

🔥 PART 3: Rest Operator (...)

👉 Opposite of spread → collects values

✓ Example

JavaScript

```
function sum(...nums) {  
  return nums.reduce((a, b) => a + b, 0);  
}
```

```
sum(1, 2, 3); // 6
```

Show more lines

✓ Object Rest

JavaScript

```
let { name, ...rest } = {  
  name: "Sameer",  
  age: 25,  
  city: "Kolkata"  
};
```

```
console.log(rest); // {age:25, city:"Kolkata"}
```

Show more lines

🔥 PART 4: Template Literals

👉 Cleaner string formatting

✓ Example

JavaScript

```
let name = "Sameer";
```

```
console.log(`Hello ${name}`);
```

Show more lines

✅ Multi-line

JavaScript

```
let text = `
```

```
Line 1
```

```
Line 2
```

```
`;
```

Show more lines

🔥 PART 5: Default Parameters

JavaScript

```
function greet(name = "Guest") {
```

```
  console.log(name);
```

```
}
```

```
greet(); // Guest
```

Show more lines

🔥 PART 6: Optional Chaining (🔥 VERY IMPORTANT)

👉 Avoid errors when value is undefined

✅ Example

JavaScript

```
let user = { name: "Sameer" };
```

```
console.log(user.address?.city); // undefined (NO ERROR ✅)
```

Show more lines

PART 7: Nullish Coalescing (??)

👉 Default value for null/undefined

JavaScript

```
let val = null;
```

```
console.log(val ?? "Default"); // Default
```

Show more lines

PART 8: Arrow Functions (Recap)

JavaScript

```
const add = (a, b) => a + b;
```

Show more lines

REAL WORLD COMBO (IMPORTANT)

JavaScript

```
let users = [  
  { name: "Sameer", age: 25 },  
  { name: "Rahul", age: 30 }  
];
```

```
let names = users.map(({ name }) => name);
```

```
console.log(names);
```

Show more lines

👉 Uses:

- map ✓
 - destructuring ✓
 - arrow function ✓
-

💡 REACT CONNECTION (VERY IMPORTANT)

👉 In React you will constantly use:

- Destructuring ✓
 - Spread ✓
 - Arrow functions ✓
 - Optional chaining ✓
-

✓ PRACTICE (VERY IMPORTANT)

🧠 Q1:

JavaScript

```
let arr = [1, 2, 3];
```

```
let [a,(a, b)];let [a, b] = arr;
```

Show more lines

🧠 Q2:

Merge arrays:

JavaScript

```
[1,2] + [3,4]
```

Show more lines

Q3:

JavaScript

```
let obj = { name: "Sameer", age: 25 };
```

```
let { name } = obj;
```

Show more lines

Q4:

Use rest operator to sum numbers

Q5:

JavaScript

```
let user = {};
```

```
console.log(user.address?.city);
```

Show more lines

MINI CHALLENGE

JavaScript

```
let products = [  
  { name: "Laptop", price: 50000 },  
  { name: "Phone", price: 20000 }  
];
```

```
// Output: ["Laptop", "Phone"]
```

..

Show more lines

👉 Use destructuring + map ✓

FINAL JS SUMMARY (IMPORTANT)

✓ Covered:

- Variables ✓
- Functions ✓
- Arrays/Objects ✓
- DOM ✓
- Closures ✓
- Async ✓
- Event Loop ✓
- this ✓
- Prototype ✓
- ES6 ✓

👉 You are now at: 💪 **Strong JavaScript Developer Level**

NEXT STEP (BIG)

👉 Now we START:

REACT (Day 11)

We will cover:

- What is React?
- Why React?

- JSX
- First componen

DAY 11 — Introduction to React + JSX + First Component

👉 Today's goal:

- Understand what React is
 - Why companies use it
 - Build your first component 
-

PART 1: What is React?

👉 React = **JavaScript Library** for building UI

💡 Developed by Facebook (Meta)

Why React?

Before React:

- DOM manipulation was messy 
 - Code was hard to manage 
-

👉 React solves this:

- ✓ Component-based architecture
 - ✓ Faster rendering (Virtual DOM)
 - ✓ Reusable code
 - ✓ Easy state management
-

◆ PART 2: Real World Idea

👉 Think like LEGO blocks 🧱

Instead of writing full page:

- Navbar component
- Footer component
- Button component

👉 Reuse everywhere ✓

🔥 PART 3: What is JSX?

👉 JSX = JavaScript + HTML

👉 Write HTML inside JavaScript

✓ Example

JavaScript

```
const element = <h1>Hello Sameer</h1>;
```

Show more lines

👉 This looks like HTML but is actually **JavaScript**

✓ Behind the scenes

JavaScript

```
React.createElement("h1", null, "Hello Sameer");
```

Show more lines

◆ JSX Rules (IMPORTANT)

✓ Must return one parent element

JavaScript

```
return (  
  <div>  
    <h1>Hello</h1>  
    <p>Welcome</p>  
  </div>  
);
```

Show more lines

✓ Close all tags

JavaScript

```

```

Show more lines

✓ Use {} for JS inside JSX

JavaScript

```
let name = "Sameer";
```

```
<h1>Hello {name}</h1>
```

Show more lines

PART 4: First React Component

👉 Component = JavaScript function returning JSX

✅ Example

```
JavaScript
function App() {
  return <h1>Hello React 🚀</h1>;
}
```

```
export default App;
Show more lines
```

👉 This is your **first component** ✅

◆ PART 5: Multiple Elements

❌ Wrong:

```
JavaScript
return (
  <h1>Hi</h1>
  <p>Text</p>
);
Show more lines
```

✅ Correct:

```
JavaScript
return (
  <div>
    <h1>Hi</h1>
  </div>
);
```

```
<p>Text</p>
</div>
);
Show more lines
```

OR

```
JavaScript
return (
  <>
    <h1>Hi</h1>
    <p>Text</p>
  </>
);
Show more lines
```

PART 6: Dynamic Data in JSX

```
JavaScript
function App() {
  let name = "Sameer";

  return <h1>Hello {name}</h1>;
}
Show more lines
```

PART 7: Simple UI Example

```
JavaScript
function App() {
  let age = 25;

  return (
    <div>
```

```
<h1>Sameer</h1>
<p>Age: {age}</p>
</div>
);
}
```

Show more lines



KEY CONCEPT

👉 React = UI = function(state → UI)



REAL REACT STRUCTURE

You will typically have:

```
None
App.js
index.js
components/
```



PRACTICE (VERY IMPORTANT)

🧠 Q1:

Create component:

JavaScript

Hello World

Show more lines

 Q2:

Display your:

- Name
 - Age
 - City
-

 Q3:

JavaScript

```
let num = 10;
```

```
// Show: "Even" or "Odd"
```

Show more lines

 Q4:

Print list:

JavaScript

```
["A", "B", "C"]
```

Show more lines

(Hint: map — from Day 2 )

MINI CHALLENGE

Create:

JavaScript

```
function App() {
```

```
  let user = {
```

```
    name: "Sameer",
```

age: 25

};

// Display both values using JSX

}

Show more lines

KEY TAKEAWAYS

- ✓ React = UI library
- ✓ JSX = HTML inside JS
- ✓ Component = Function returning UI
- ✓ `{}` = run JS inside JSX

DAY 12 — Props (VERY IMPORTANT)

👉 Props = **data passed from one component to another**

💡 Think: 👉 **Parent** → **Child** communication

PART 1: Why Props?

👉 Without props:

- Components are static ❌

👉 With props:

- Components become dynamic ✓
- Reusable ✓

◆ PART 2: Basic Props Example

✓ Step 1: Create Child Component

JavaScript

```
function Greeting(props) {  
  return <h1>Hello {props.name}</h1>;  
}
```

Show more lines

✓ Step 2: Use it in Parent

JavaScript

```
function App() {  
  return <Greeting name="Sameer" />;  
}
```

Show more lines

👉 Output:

None

Hello Sameer

🔥 PART 3: Multiple Props

JavaScript

```
function User(props) {  
  return (  
    <div>  
      <h1>{props.name}</h1>  
    </div>  
  )  
}
```

```
<p>{props.age}</p>
</div>
);
}
```

```
function App() {
  return <User name="Sameer" age={25} />;
}
```

Show more lines

◆ PART 4: Destructuring Props (IMPORTANT 🔥)

👉 Cleaner way

```
JavaScript
function User({ name, age }) {
  return (
    <div>
      <h1>{name}</h1>
      <p>{age}</p>
    </div>
  );
}
```

Show more lines

🔥 PART 5: Props with Dynamic Data

```
JavaScript
function App() {
  let user = {
    name: "Sameer",
```


age: 25

};

return <User name={user.name} age={user.age} />;

}

Show more lines

◆ PART 6: Passing Arrays

JavaScript

```
function List({ items }) {
```

```
  return (
```

```
    <ul>
```

```
    {items.map((item) => (
```

```
      <li>{item}</li>
```

```
    ))}
```

```
    </ul>
```

```
  );
```

```
}
```

```
function App() {
```

```
  return <List items={['A', 'B', 'C']} />;
```

```
}
```

Show more lines

🔥 PART 7: Passing Functions (VERY IMPORTANT 🔥)

👉 Used for event handling

JavaScript

```
function Button({ handleClick }) {
```

```
  return <button onClick={handleClick}>Click</button>;
```

```
}  
  
function App() {  
  function show() {  
    console.log("Clicked!");  
  }  
  
  return <Button handleClick={show} />;  
}
```

Show more lines

PART 8: Props are Read-Only

 Wrong:

JavaScript
props.name = "Rahul"; // 
Show more lines

 Correct:

- Props cannot be changed
 - Only read and use
-

PART 9: Real World Example

JavaScript

```
function Product({ name, price }) {  
  return (  
    <div>  
      <h2>{name}</h2>  
      <p>₹{price}</p>  
    </div>  
  );  
}
```

```
}
```

```
function App() {  
  return (  
    <div>  
      <Product name="Laptop" price={50000} />  
      <Product name="Phone" price={20000} />  
    </div>  
  );  
}
```

Show more lines

👉 Reusable component ✅



KEY FLOW

None

Parent → passes props → Child uses props



REACT INTERVIEW GOLD

👉 Props vs State

Props	State
Passed from parent	Managed inside component
Read-only	Mutable
Used for data flow	Used for dynamic UI



PRACTICE (VERY IMPORTANT)

Q1:

Create component:

JavaScript

```
<Welcome name="Sameer" />
```

Show more lines

Q2:

Pass:

- name
 - age
 - city
-

Q3:

Pass array & display list

Q4:

Pass function as prop and trigger click



MINI CHALLENGE

 Build this:

JavaScript

```
function Card({ title, desc }) {
```

```
// display both
}  
  
function App() {  
  return (  
    <>  
    <Card title="React" desc="Frontend Library" />  
    <Card title="JS" desc="Programming Language" />  
    </>  
  );  
}
```

Show more lines

KEY TAKEAWAY

- 👉 Props = data passing mechanism
- 👉 Makes components reusable
- 👉 Used everywhere in React

DAY 13 — State & **useState** (🔥 MOST IMPORTANT 🔥)

- 👉 If you understand this well: ☒ You can build real apps
 - ☒ You understand React properly
-

PART 1: What is State?

- 👉 State = **data that can change over time**

💡 Example:

- Counter value ✓
- Input text ✓
- API data ✓

✓ Difference from Props

Props	State
Passed from parent	Managed inside component
Read-only	Can change ✓

◆ PART 2: What is **useState**?

👉 Hook used to **create state in functional components**

✓ Syntax

JavaScript

```
const [state, setState] = useState(initialValue);
```

Show more lines

👉 Example:

JavaScript

```
const [count, setCount] = useState(0);
```

Show more lines

- **count** → value
- **setCount** → function to update

PART 3: Basic Counter Example

JavaScript

```
import { useState } from "react";
```

```
function App() {  
  const [count, setCount] = useState(0);
```

```
  return (  
    <div>  
      <h1>{count}</h1>
```

```
      <button onClick={() => setCount(count + 1)}>
```

```
        Increase
```

```
      </button>
```

```
    </div>
```

```
  );
```

```
}
```

Show more lines

👉 Flow:

1. UI shows `count = 0`
 2. Click → updates state
 3. React re-renders 
-

◆ PART 4: Why not just use variable?

JavaScript

```
let count = 0; // ❌ WRONG
```

Show more lines

👉 UI will NOT update ❌

💡 Only `useState` triggers re-render ✅

🔥 PART 5: Multiple States

JavaScript

```
function App() {  
  const [name, setName] = useState("Sameer");  
  const [age, setAge] = useState(25);  
  
  return (  
    <div>  
      <h1>{name}</h1>  
      <p>{age}</p>  
    </div>  
  );  
}
```

Show more lines

♦ PART 6: Updating State (IMPORTANT



✅ Correct way

JavaScript

```
setCount(count + 1);
```

Show more lines

✅ Functional Update (BEST PRACTICE 🔥)

JavaScript

```
setCount(prev => prev + 1);
```

Show more lines

👉 Use when:

- multiple updates
- async operations

PART 7: Input Handling (REAL WORLD

)

JavaScript

```
function App() {  
  const [name, setName] = useState("");  
  
  return (  
    <div>  
      <input  
        type="text"  
        onChange={(e) => setName(e.target.value)}  
      />  
  
      <h1>Hello {name}</h1>  
    </div>  
  );  
}
```

Show more lines

👉 This is used in: ☒ Forms

☒ Search bars

☒ Login pages

◆ PART 8: State with Objects

JavaScript

```
const [user, setUser] = useState({  
  name: "Sameer",  
  age: 25  
});
```

Show more lines

✓ Update Object (IMPORTANT)

JavaScript

```
setUser({ ...user, age: 26 });
```

Show more lines

👉 Spread operator is needed ✓

◆ PART 9: State with Arrays

JavaScript

```
const [items, setItems] = useState([1, 2, 3]);
```

Show more lines

✓ Add item

JavaScript

```
setItems([...items, 4]);
```

Show more lines

✓ Remove item

JavaScript

```
setItems(items.filter(item => item !== 2));
```

Show more lines

PART 10: Real Mini App (Counter)

JavaScript

```
function App() {  
  const [count, setCount] = useState(0);  
  
  return (  
    <div>  
      <h1>{count}</h1>  
  
      <button onClick={() => setCount(count + 1)}>+</button>  
      <button onClick={() => setCount(count - 1)}>-</button>  
    </div>  
  );  
}
```

Show more lines

VERY IMPORTANT RULE

👉 NEVER mutate state directly ❌

JavaScript

```
user.age = 30; // ❌ WRONG
```

Show more lines

✅ Always use setter:

JavaScript

```
setUser({ ...user, age: 30 });
```

Show more lines



REACT CORE PRINCIPLE

👉 State changes → UI updates automatically



PRACTICE (VERY IMPORTANT)

🧠 Q1:

Create counter:

- Increment
 - Decrement
-

🧠 Q2:

Create input field

- Display typed value
-

🧠 Q3:

Store array in state

- Add item
 - Remove item
-

🧠 Q4:

Store object in state

- Update one field

MINI CHALLENGE

👉 Build this:

```
JavaScript
function App() {
  const [count, setCount] = useState(0);

  // When count is even → show "Even"
  // else → "Odd"
}
Show more lines
```

INTERVIEW GOLD

👉 Question: Why useState?

✅ Answer: 👉 "useState allows functional components to manage dynamic data and triggers re-render when state changes."

FINAL UNDERSTANDING

👉 Think:

None

State = Brain of component 🧠

UI = Face 👤

👉 When brain changes → face updates automatically ✅

DAY 14 — **useEffect** Hook (VERY IMPORTANT)

PART 1: What is **useEffect**?

👉 **useEffect** is used to handle **side effects**

💡 Side effects = things that happen **outside UI rendering**

Examples:

- ✅ API calls
 - ✅ Fetching data
 - ✅ Timers (**setTimeout**)
 - ✅ Updating DOM manually
-

◆ PART 2: Basic Syntax

JavaScript

```
import { useEffect } from "react";
```

```
useEffect(() => {  
  // code runs here  
}, []);
```

Show more lines

👉 Structure:

- First → function (what to run)
 - Second → dependency array (when to run)
-

PART 3: Types of useEffect

✓ 1. Run ONLY once (Component Mount)

JavaScript
useEffect(() => {
 console.log("Component Loaded");
}, []);
[Show more lines](#)

👉 Runs once when page loads ✓

✓ 2. Run on state change

JavaScript
useEffect(() => {
 console.log("Count changed");
}, [count]);
[Show more lines](#)

👉 Runs whenever `count` changes ✓

✓ 3. Run on every render

JavaScript
useEffect(() => {
 console.log("Runs every time");
});

Show more lines

👉 No dependency array → runs always ❌ (use carefully)

🔥 PART 4: Example with Counter

JavaScript

```
import { useState, useEffect } from "react";

function App() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    console.log("Count updated:", count);
  }, [count]);

  return (
    <div>
      <h1>{count}</h1>
      <button onClick={() => setCount(count + 1)}>+</button>
    </div>
  );
}
```

Show more lines

👉 Flow:

1. User clicks → state changes
 2. Component re-renders
 3. `useEffect` runs ✅
-

🔥 PART 5: API Call (REAL WORLD 🔥)

JavaScript

```
import { useState, useEffect } from "react";
```

```
function App() {
```

```
  const [users, setUsers] = useState([]);
```

```
  useEffect(() => {
```

```
    fetch("https://jsonplaceholder.typicode.com/users")
```

```
    .then((res) => res.json())
```

```
    .then((data) => setUsers(data));
```

```
  }, []);
```

```
  return (
```

```
    <div>
```

```
    {users.map((user) => (
```

```
      <p key={user.id}>{user.name}</p>
```

```
    ))}
```

```
    </div>
```

```
  );
```

```
}
```

Show more lines

👉 This is EXACTLY how real apps work ✅

PART 6: Async/Await Version (BEST PRACTICE)

JavaScript

```
useEffect(() => {
```

```
  async function fetchData() {
```

```
    let res = await fetch("https://jsonplaceholder.typicode.com/users");
```

```
    let data = await res.json();
```

```
    setUsers(data);
```

```
}
```

```
fetchData();
```

```
}, []);
```

Show more lines

PART 7: Cleanup Function (VERY IMPORTANT)

👉 Used to clean resources

Example (Timer)

JavaScript

```
useEffect(() => {
```

```
  let timer = setInterval(() => {
```

```
    console.log("Running...");
```

```
  }, 1000);
```

```
  return () => {
```

```
    clearInterval(timer);
```

```
  };
```

```
}, []);
```

Show more lines

👉 Runs:

- Setup → when mounted
 - Cleanup → when unmounted 
-

PART 8: Common Mistake

✗ Infinite loop:

```
JavaScript
useEffect(() => {
  setCount(count + 1);
}, [count]);
```

Show more lines

👉 Causes infinite re-render ✗

PART 9: REAL LIFE ANALOGY

👉 Think like:

- `useEffect` = background worker 
 - UI render = main job
-

PART 10: Key Rules

- ✓ Always define dependencies
 - ✓ Use empty array for one-time execution
 - ✓ Avoid unnecessary re-renders
 - ✓ Use cleanup when needed
-



REACT MENTAL MODEL

None

Render → useEffect runs → state updates → re-render



PRACTICE (VERY IMPORTANT)

 **Q1:**

Run code only once when component loads

 **Q2:**

Run effect only when `count` changes

 **Q3:**

Fetch API and display data

 **Q4:**

Create timer using `setInterval`



MINI CHALLENGE

 Build:

JavaScript

```
function App() {  
  const [count, setCount] = useState(0);  
  
  // Print "Even" or "Odd" in console using useEffect  
}
```

Show more lines

INTERVIEW GOLD

👉 Question: What is useEffect?

✅ Answer: 👉 *"useEffect is a React hook used to handle side effects like API calls, subscriptions, and manual DOM updates in functional components."*

FINAL UNDERSTANDING

👉 Before:

- Class lifecycle methods (componentDidMount etc.)

👉 Now:

- `useEffect` replaces all ✅

DAY 15 — React Forms & Controlled Components

PART 1: What is a Controlled Component?

👉 Controlled = **React controls the input**

💡 Meaning:

- Input value is stored in **state**
- React updates UI 

Basic Example

JavaScript

```
import { useState } from "react";
```

```
function App() {  
  const [name, setName] = useState("");  
  
  return (  
    <div>  
      <input  
        type="text"  
        value={name}  
        onChange={(e) => setName(e.target.value)}  
      />  
  
      <h1>{name}</h1>  
    </div>  
  );  
}
```

Show more lines

👉 Flow:

1. User types
 2. `onChange` triggers
 3. State updates
 4. UI updates 
-

PART 2: Why Controlled Components?

-  React has full control
 -  Easy validation
 -  Better data handling
-

◆ PART 3: Multiple Inputs (IMPORTANT)

Example Form

```
JavaScript
function App() {
  const [form, setForm] = useState({
    name: "",
    email: ""
  });

  function handleChange(e) {
    const { name, value } = e.target;

    setForm({
      ...form,
      [name]: value
    });
  }
}
```

```
return (  
<div>  
<input  
  name="name"  
  value={form.name}  
  onChange={handleChange}  
  placeholder="Name"  
>  
  
<input  
  name="email"  
  value={form.email}  
  onChange={handleChange}  
  placeholder="Email"  
>  
  
<h2>{form.name}</h2>  
<h2>{form.email}</h2>  
</div>  
);  
}
```

Show more lines

👉 Key concept:

JavaScript

[name]: value

Show more lines

✅ Dynamically updates field

PART 4: Form Submit

JavaScript


```
function App() {
  const [name, setName] = useState("");

  function handleSubmit(e) {
    e.preventDefault(); // IMPORTANT 🔥
    console.log(name);
  }

  return (
    <form onSubmit={handleSubmit}>
      <input
        value={name}
        onChange={(e) => setName(e.target.value)}
      />

      <button type="submit">Submit</button>
    </form>
  );
}
```

Show more lines

👉 `e.preventDefault()`

✅ Prevents page reload

◆ PART 5: Checkbox Example

JavaScript

```
const [checked, setChecked] = useState(false);

<input
  type="checkbox"
  checked={checked}
  onChange={(e) => setChecked(e.target.checked)}
/>
```

/>

Show more lines

◆ PART 6: Select Dropdown

JavaScript

```
const [city, setCity] = useState("");
```

```
<select onChange={(e) => setCity(e.target.value)}>
```

```
<option value="Kolkata">Kolkata</option>
```

```
<option value="Delhi">Delhi</option>
```

```
</select>
```

Show more lines

🔥 PART 7: Simple Form Validation

JavaScript

```
if (name === "") {
```

```
  alert("Name is required");
```

```
}
```

Show more lines

🔥 PART 8: Real Mini App (Login Form)

JavaScript

```
function App() {
```

```
  const [form, setForm] = useState({
```

```
    email: "",
```

```
    password: ""
```

```
  });
```

```
  function handleChange(e) {
```

```
setForm({ ...form, [e.target.name]: e.target.value });  
}
```

```
function handleSubmit(e) {  
  e.preventDefault();  
  console.log(form);  
}
```

```
return (  
  <form onSubmit={handleSubmit}>  
    <input  
      name="email"  
      onChange={handleChange}  
      placeholder="Email"  
    />
```

```
    <input  
      name="password"  
      type="password"  
      onChange={handleChange}  
      placeholder="Password"  
    />
```

```
    <button type="submit">Login</button>  
  </form>  
);  
}
```

Show more lines



PART 9: Controlled vs Uncontrolled

Controlled	Uncontrolled
Uses state 	Uses DOM 

React controls	Manual access
Recommended 	Rare



KEY CONCEPT

👉 React wants:

None

Input → State → UI



PRACTICE (VERY IMPORTANT)

🧠 Q1:

Create input field
Display typed value

🧠 Q2:

Create form with:

- name
 - email
-

🧠 Q3:

On submit → print data

 Q4:

Add validation:

- empty input → error
-

MINI CHALLENGE

 Build:

JavaScript

Form:

- Name
- Age

On submit:

- Show data below form

Show more lines

INTERVIEW GOLD

 Question: Controlled component?

 Answer:  "A controlled component is a form element whose value is controlled by React state."

FINAL UNDERSTANDING

 Think:

None

User types → React stores → React shows

DAY 16 — Lifting State Up + Component Communication

👉 This is used when: ☒ Multiple components need same data

☒ Parent & child communication

☒ Building real apps (forms, dashboards, etc.)

PART 1: The Problem

👉 Example:

You have:

None

Parent

├— Child1 (input)

└— Child2 (display)

👉 Problem:

- Input in Child1 ❌
- Display in Child2 ❌

👉 They cannot directly talk to each other

SOLUTION → Lifting State Up

👉 Move shared state to **common parent**

◆ PART 2: Basic Concept

👉 Instead of storing state in child 👉

✗ Wrong:

JavaScript

```
function Child1() {  
  const [name, setName] = useState("");  
}
```

Show more lines

✓ Correct: 👉 Move state to parent

JavaScript

```
function App() {  
  const [name, setName] = useState("");  
}
```

Show more lines

👉 Then pass:

- Data → via props
 - Functions → via props
-

PART 3: Example (IMPORTANT)

Parent

JavaScript

```
import { useState } from "react";

function App() {
  const [name, setName] = useState("");

  return (
    <div>
      <ChildInput setName={setName} />
      <ChildDisplay name={name} />
    </div>
  );
}
```

Show more lines

Child 1 (Input)

JavaScript

```
function ChildInput({ setName }) {
  return (
    <input
      onChange={(e) => setName(e.target.value)}
      placeholder="Enter name"
    />
  );
}
```

Show more lines

Child 2 (Display)

JavaScript

```
function ChildDisplay({ name }) {
```



```
return <h1>{name}</h1>;  
}
```

Show more lines

👉 Flow:

None

ChildInput → updates parent state

Parent → passes updated state to ChildDisplay

✅ This is called **lifting state up**

🔥 PART 4: Key Rule

👉 Data flows:

None

Parent → Child ✅ (one-way)

👉 To update parent: 👉 Child sends function ✅

◆ PART 5: Real World Analogy

👉 Think:

- Parent = Manager 👤
- Children = Employees 🏢

👉 Employees don't talk directly

👉 Everything goes through manager ✅

PART 6: Another Example (Counter Control)

Parent

JavaScript

```
function App() {  
  const [count, setCount] = useState(0);  
  
  return (  
    <>  
    <CounterButtons setCount={setCount} />  
    <Display count={count} />  
    </>  
  );  
}
```

Show more lines

Child Buttons

JavaScript

```
function CounterButtons({ setCount }) {  
  return (  
    <>  
    <button onClick={() => setCount((c) => c + 1)}>+</button>  
    <button onClick={() => setCount((c) => c - 1)}>-</button>  
    </>  
  );  
}
```

Show more lines

✓ Child Display

JavaScript

```
function Display({ count }) {  
  return <h1>{count}</h1>;  
}
```

Show more lines

🔥 PART 7: Why This is IMPORTANT

✓ Used in:

- Forms ✓
 - Search features ✓
 - Dashboards ✓
 - Shared data ✓
-

🔥 PART 8: Common Mistake ⚠️

✗ Trying to share state directly between children

👉 Always: ✓ Lift to parent

🔥 PART 9: Pattern Name

👉 This is also called: ✓ **State Lifting Pattern**



KEY MINDSET

👉 Whenever you think:

? "Two components need same data?"

✅ Answer: 👉 **Lift state up to parent**

✅ PRACTICE (VERY IMPORTANT)

🧠 Q1:

Create:

- Input component
 - Display component
 - 👉 Share value
-

🧠 Q2:

Create counter:

- Buttons in one component
 - Display in another
-

🧠 Q3:

Form + Result display (two components)

🎯 MINI CHALLENGE

👉 Build:

None

App

└─ Input (enter age)

└─ Result (show Adult / Minor)

✅ Logic:

- Age < 18 → Minor
- Else → Adult

🔥 INTERVIEW GOLD

👉 Question: What is lifting state up?

✅ Answer: 👉 *"Lifting state up is the process of moving shared state to the closest common parent so multiple components can access and update it."*

🔥 FINAL UNDERSTANDING

👉 Think:

None

Shared data → belongs to parent

Child → only uses or updates it

🚀 NEXT DAY (DAY 17)

🔥 Context API (VERY IMPORTANT 🔥)

👉 Avoid prop drilling

👉 Used in real large apps

✅ YOUR NEXT STEP

Reply: 👉 "Next Day"

Now you're thinking like a

💪 **REAL REACT DEVELOPER** 🔥🚀

Sources

Provide your feedback on BizChat

You said:

start day 17

Copilot said:

Copilot

Perfect 🔥 Now you are entering **advanced React architecture**

👉 This topic is used in **real production apps**

🚀 DAY 17 — Context API (🔥 VERY IMPORTANT 🔥)

👉 This solves a BIG problem in React:

🔥 PART 1: The Problem — Prop Drilling 😓

👉 Prop drilling = passing props through many levels

Example:

None

App → Child1 → Child2 → Child3

👉 If Child3 needs data:

JavaScript

```
<App user={user}>
  <Child1 user={user}>
    <Child2 user={user}>
      <Child3 user={user} />
    </Child2>
  </Child1>
</App>
```

Show more lines

- ✗ Messy
- ✗ Hard to maintain
- ✗ Not scalable

✅ SOLUTION → Context API

👉 Context = global storage for your app ✅

👉 Any component can access data directly

👉 No need for prop drilling

◆ PART 2: Concept

👉 Context = like a **global variable inside React**

🔥 PART 3: Steps to Use Context

👉 3 main steps:

✅ Step 1: Create Context

JavaScript

```
import { createContext } from "react";
```

```
const UserContext = createContext();
```

Show more lines

✅ Step 2: Provide Context

👉 Wrap components with provider

JavaScript

```
function App() {
```

```
  return (
```

```
    <UserContext.Provider value="Sameer">
```

```
      <Child />
```

```
    </UserContext.Provider>
```

```
  );
```

```
}
```

Show more lines

✅ Step 3: Consume Context

👉 Inside any child component:

JavaScript

```
import { useContext } from "react";
```

```
function Child() {  
  const user = useContext(UserContext);
```

```
  return <h1>{user}</h1>;  
}
```

Show more lines

👉 Output:

None

Sameer ✅

🔥 PART 4: Real Example (IMPORTANT 🔥)

✅ Context File

JavaScript

```
import { createContext } from "react";
```

```
export const ThemeContext = createContext();
```

Show more lines

✅ App.js

JavaScript

```
import { ThemeContext } from "../ThemeContext";
```

```
function App() {  
  return (  
    <ThemeContext.Provider value="dark">  
      <Navbar />  
    </ThemeContext.Provider>  
  );  
}
```

Show more lines

Child Component

JavaScript

```
import { useContext } from "react";  
import { ThemeContext } from "../ThemeContext";
```

```
function Navbar() {  
  const theme = useContext(ThemeContext);
```

```
  return <h1>{theme}</h1>;  
}
```

Show more lines

PART 5: Passing Objects (IMPORTANT)

JavaScript

```
<UserContext.Provider value={{ name: "Sameer", age: 25 }}>
```

Show more lines

Use

JavaScript

```
const user = useContext(UserContext);
```

```
console.log(user.name);
```

Show more lines

PART 6: Updating Context (ADVANCE **)**

👉 You can pass state + function

JavaScript

```
function App() {  
  const [user, setUser] = useState("Sameer");  
  
  return (  
    <UserContext.Provider value={{ user, setUser }}>  
    <Child />  
    </UserContext.Provider>  
  );  
}
```

Show more lines

Child Update

JavaScript

```
function Child() {  
  const { user, setUser } = useContext(UserContext);  
  
  return (  
    <button onClick={() => setUser("Rahul")}>  
      Change Name  
    </button>  
  );  
}
```

Show more lines

PART 7: When to Use Context?

- ✓ Authentication (user login)
 - ✓ Theme (dark/light mode)
 - ✓ Language preferences
 - ✓ Global settings
-

PART 8: When NOT to Use?

- ✗ Too much context → performance issues
 - ✗ Large apps → use Redux instead
-

PART 9: Context vs Props

Props	Context
Passed manually	Global access
Simple apps	Large apps
Easy	Slightly complex

PART 10: Real Analogy

- 👉 Context = WhatsApp group 📱
- 👉 Everyone in group can read messages ✓
- 👉 No need to send individually



KEY RULE

👉 Use Context when: ☒ Many components need same data



PRACTICE (VERY IMPORTANT)

🧠 Q1:

Create:

- Context
 - Provide value "Hello"
 - Display in child
-

🧠 Q2:

Pass object:

JavaScript

{name, age}

Show more lines

🧠 Q3:

Update context value from child

MINI CHALLENGE

👉 Build:

None

Theme Toggle

- Dark / Light
- Use Context

INTERVIEW GOLD

👉 Question: What is Context API?

✅ Answer: 👉 *"Context API is a React feature used to share global data across components without passing props manually at every level."*

FINAL UNDERSTANDING

👉 Think:

None

Context = Global state

Props = Local data passing

DAY 18 — React Router ( VERY IMPORTANT )

- 👉 React is a **single-page application (SPA)**
 - 👉 It doesn't reload pages like traditional websites
 - 👉 So how does navigation work?
 - ✅ Using React Router
-

🔥 PART 1: What is React Router?

- 👉 React Router = library for **navigation between pages/components**

💡 Instead of reloading pages:

None
`/home → /about → /contact`

- 👉 React just switches components ✅
-

◆ PART 2: Install React Router

Shell

```
npm install react-router-dom
```

Show more lines

🔥 PART 3: Basic Setup

✅ Step 1: Wrap App

JavaScript

```
import { BrowserRouter } from "react-router-dom";
```

```
function App() {  
  return (  
    <BrowserRouter>  
    <h1>My App</h1>  
    </BrowserRouter>  
  );  
}
```

Show more lines

Step 2: Create Routes

JavaScript

```
import { Routes, Route } from "react-router-dom";
```

```
function App() {  
  return (  
    <BrowserRouter>  
    <Routes>  
    <Route path="/" element={<Home />} />  
    <Route path="/about" element={<About />} />  
    </Routes>  
    </BrowserRouter>  
  );  
}
```

Show more lines

Step 3: Components

JavaScript

```
function Home() {  
  return <h1>Home Page</h1>;  
}
```

```
function About() {  
  return <h1>About Page</h1>;  
}
```



```
}
```

Show more lines

🔥 PART 4: Navigation (IMPORTANT 🔥)

👉 Use `<Link>` instead of `<a>`

✅ Example

JavaScript

```
import { Link } from "react-router-dom";
```

```
function Navbar() {  
  return (  
    <>  
    <Link to="/">Home</Link>  
    <Link to="/about">About</Link>  
    </>  
  );  
}
```

Show more lines

👉 Why NOT `<a>` tag?

- `<a>` reloads page ❌
 - `<Link>` switches view ✅
-

🔥 PART 5: Full Example

JavaScript

```
function App() {  
  return (  
    <BrowserRouter>  
  
    <Navbar />  
  
    <Routes>  
    <Route path="/" element={<Home />} />  
    <Route path="/about" element={<About />} />  
    </Routes>  
  
  </BrowserRouter>  
);  
}  
Show more lines
```

🔥 PART 6: useNavigate (🔥 IMPORTANT)

👉 Navigate programmatically

✅ Example

```
JavaScript  
import { useNavigate } from "react-router-dom";  
  
function Home() {  
  const navigate = useNavigate();  
  
  return (  
    <button onClick={() => navigate("/about")}>  
      Go to About  
    </button>  
  );  
}
```

Show more lines

👉 Used for: ☒ Login redirect

☒ After form submit

PART 7: Dynamic Routes

👉 Useful for:

- Product pages
 - User profiles
-

☒ Example

JavaScript

```
<Route path="/user/:id" element={<User />} />
```

Show more lines

☒ Access Param

JavaScript

```
import { useParams } from "react-router-dom";
```

```
function User() {  
  const { id } = useParams();
```

```
  return <h1>User ID: {id}</h1>;  
}
```

Show more lines

👉 URL:

None

/user/101

👉 Output:

None

User ID: 101

PART 8: 404 Page

JavaScript

```
<Route path="" element={<NotFound />} />
```

Show more lines

PART 9: Nested Routing (ADVANCED)

JavaScript

```
<Route path="/dashboard" element={<Dashboard />}>
```

```
<Route path="profile" element={<Profile />} />
```

```
</Route>
```

Show more lines

PART 10: Real Example Structure

None

App

├─ Navbar

├─ Routes

| └─ Home

```
|   |— About
|   |— Contact
|   |— User
```



KEY CONCEPT

👉 React Router =

- ✓ Switch between components
 - ✓ No page reload
 - ✓ Smooth experience
-



PRACTICE (VERY IMPORTANT)

🧠 Q1:

Create 2 pages:

- Home
 - About
-

🧠 Q2:

Add navigation using [Link](#)

🧠 Q3:

Create button → navigate to new page

 **Q4:**

Create dynamic route:

```
None
/user/1
/user/2
```

MINI CHALLENGE

 Build:

```
None
Navbar :
- Home
- Contact

Pages :
- Home page text
- Contact page text
```

INTERVIEW GOLD

 Question: What is React Router?

✓ Answer:  "React Router is a library used to handle navigation in React applications without reloading the page."

FINAL UNDERSTANDING

👉 Think:

None

URL change → Component change → No reload

DAY 19 — API Integration in React

👉 Goal: ☒ Fetch data from API

☒ Display in UI

☒ Handle loading & errors

PART 1: What is API?

👉 API = bridge between frontend & backend

💡 Example: React → API → Database

👉 Example API:

None

<https://jsonplaceholder.typicode.com/users>

◆ PART 2: Basic Fetch in React

JavaScript

```
import { useState, useEffect } from "react";
```

```
function App() {
```

```
  const [users, setUsers] = useState([]);
```

```
  useEffect(() => {
```

```
    fetch("https://jsonplaceholder.typicode.com/users")
```

```
    .then((res) => res.json())
```

```
    .then((data) => setUsers(data));
```

```
  }, []);
```

```
  return (
```

```
    <div>
```

```
    {users.map((user) => (
```

```
      <p key={user.id}>{user.name}</p>
```

```
    ))}
```

```
    </div>
```

```
  );
```

```
}
```

Show more lines

👉 Flow:

1. Component loads ✓
 2. useEffect runs ✓
 3. Fetch API ✓
 4. Store in state ✓
 5. Display UI ✓
-

PART 3: Using Async/Await (BEST PRACTICE)

JavaScript

```
useEffect(() => {  
  async function fetchData() {  
    let res = await fetch("https://jsonplaceholder.typicode.com/users");  
    let data = await res.json();  
    setUsers(data);  
  }  
}
```

fetchData();

}, []);

Show more lines

PART 4: Loading State (VERY IMPORTANT)

👉 Show loader while fetching

Example

JavaScript

```
const [loading, setLoading] = useState(true);
```

```
useEffect(() => {  
  async function fetchData() {  
    let res = await fetch("https://jsonplaceholder.typicode.com/users");  
    let data = await res.json();
```

```
    setUsers(data);
```

```
    setLoading(false);
```

```
  }
```

fetchData();

}, []);

Show more lines



JavaScript

```
if (loading) {  
  return <h1>Loading...</h1>;  
}
```

Show more lines

PART 5: Error Handling

JavaScript

```
const [error, setError] = useState(null);
```

```
useEffect(() => {  
  async function fetchData() {  
    try {  
      let res = await fetch("wrong-url");  
      let data = await res.json();  
      setUsers(data);  
    } catch (err) {  
      setError("Failed to fetch data");  
    } finally {  
      setLoading(false);  
    }  
  }  
}
```

```
fetchData();
```

```
}, []);
```

Show more lines



JavaScript

```
if (error) {  
  return <h1>{error}</h1>;  
}
```

Show more lines

PART 6: Full Production Example

JavaScript

```
function App() {  
  const [users, setUsers] = useState([]);  
  const [loading, setLoading] = useState(true);  
  const [error, setError] = useState(null);  
  
  useEffect(() => {  
    async function fetchData() {  
      try {  
        let res = await fetch("https://jsonplaceholder.typicode.com/users");  
        let data = await res.json();  
        setUsers(data);  
      } catch {  
        setError("Error loading data");  
      } finally {  
        setLoading(false);  
      }  
    }  
  
    fetchData();  
  }, []);  
  
  if (loading) return <h1>Loading...</h1>;  
  if (error) return <h1>{error}</h1>;  
  
  return (  
    <div>
```

```
{users.map((user) => (  
  <div key={user.id}>  
    <h2>{user.name}</h2>  
    <p>{user.email}</p>  
  </div>  
))}  
</div>  
);  
}
```

Show more lines

PART 7: Fetch vs Axios (Interview)

👉 Fetch:

- Built-in 
- Less features

👉 Axios:

- External library
 - Better error handling 
 - Automatic JSON parsing 
-

Axios Example

JavaScript

```
import axios from "axios";
```

```
useEffect(() => {  
  async function fetchData() {  
    let res = await axios.get(  
      "https://jsonplaceholder.typicode.com/users"  
    );  
    setUsers(res.data);  
  }  
});
```

```
}
```

```
fetchData();
```

```
}, []);
```

Show more lines

PART 8: Real Use Cases

- ✓ User list
 - ✓ Product listing
 - ✓ Dashboard data
 - ✓ Weather app
-

PART 9: Optimization (IMPORTANT)

👉 Avoid multiple API calls:

JavaScript

```
useEffect(() => {
```

```
fetchData();
```

```
}, []); // ✓ only once
```

Show more lines



REACT DATA FLOW

None

API → State → UI



PRACTICE (VERY IMPORTANT)

Q1:

Fetch users

Display name

Q2:

Show loading state

Q3:

Handle error

Q4:

Display:

- name
 - email
 - phone
-



MINI CHALLENGE

 Build:

None

User Card UI

- Name
- Email
- City

👉 From API ☒

INTERVIEW GOLD

👉 Question: How do you fetch data in React?

☒ Answer: 👉 *"Using `useEffect` to call APIs and `useState` to store and render data."*

FINAL UNDERSTANDING

👉 Think:

None

App loads → API call → Data stored → UI updates

DAY 20 — Custom Hooks (ADVANCED + REAL WORLD)

👉 This is about: ☒ Code reusability

☒ Clean architecture

☒ Writing professional React code

🔥 PART 1: What is a Custom Hook?

👉 Custom Hook = **your own reusable function using React hooks**

💡 Think: 👉 Instead of repeating logic → reuse it ✅

◆ Naming Rule

👉 Always start with **use**

✅ Examples:

```
None  
useFetch  
useAuth  
useForm
```

🔥 PART 2: Why Custom Hooks?

👉 Problem:

```
JavaScript  
useEffect(() => {  
  fetchData();  
}, []);  
Show more lines
```

👉 If you use this in multiple components ❌
→ duplicate code

✅ Solution: 👉 Move logic into custom hook

PART 3: First Custom Hook

✅ Create Hook

JavaScript

```
import { useState, useEffect } from "react";
```

```
function useFetch(url) {  
  const [data, setData] = useState([]);
```

```
  useEffect(() => {  
    async function fetchData() {  
      let res = await fetch(url);  
      let result = await res.json();  
      setData(result);  
    }  
  }, [url]);
```

```
  return data;  
}
```

Show more lines

✅ Use in Component

JavaScript

```
function App() {  
  const users = useFetch("https://jsonplaceholder.typicode.com/users");
```

```
return (  
  <div>  
    {users.map((user) => (  
      <p key={user.id}>{user.name}</p>  
    ))}  
  </div>  
);  
}
```

Show more lines

👉 🎯 Now reusable anywhere ✅

🔥 PART 4: Add Loading + Error (PRO LEVEL 🔥)

JavaScript

```
function useFetch(url) {  
  const [data, setData] = useState([]);  
  const [loading, setLoading] = useState(true);  
  const [error, setError] = useState(null);
```

```
  useEffect(() => {  
    async function fetchData() {  
      try {  
        let res = await fetch(url);  
        let result = await res.json();  
        setData(result);  
      } catch {  
        setError("Error fetching data");  
      } finally {  
        setLoading(false);  
      }  
    }  
  })
```

```
}  
  
fetchData();  
, [url]);  
  
return { data, loading, error };  
}
```

Show more lines

Use

```
JavaScript  
function App() {  
  const { data, loading, error } = useFetch("API_URL");  
  
  if (loading) return <h1>Loading...</h1>;  
  if (error) return <h1>{error}</h1>;  
  
  return (  
    <div>  
      {data.map((item) => (  
        <p key={item.id}>{item.name}</p>  
      ))}  
    </div>  
  );  
}
```

Show more lines

PART 5: Custom Hook for Form (REAL WORLD)

JavaScript

```
function useForm(initialState) {  
  const [form, setForm] = useState(initialState);
```

```
  
  function handleChange(e) {  
    setForm({  
      ...form,  
      [e.target.name]: e.target.value  
    });  
  }  
  
  return { form, handleChange };  
}
```

Show more lines

Use

JavaScript

```
function App() {  
  const { form, handleChange } = useForm({  
    name: "",  
    email: ""  
  });  
  
  return (  
    <>  
    <input name="name" onChange={handleChange} />  
    <input name="email" onChange={handleChange} />  
  
    <h1>{form.name}</h1>  
    </>  
  );  
}
```

Show more lines

PART 6: Rules of Hooks (VERY IMPORTANT)

👉 Must follow:

- ✓ Only call hooks at top level
 - ✓ Only call inside React function or custom hook
-

Wrong

```
JavaScript
if (condition) {
  useState(0); // 
}
```

Show more lines

PART 7: Real Use Cases

👉 You will create hooks for:

- ✓ API fetching (`useFetch`)
 - ✓ Authentication (`useAuth`)
 - ✓ Forms (`useForm`)
 - ✓ Theme (`useTheme`)
-

PART 8: Folder Structure (PRO LEVEL)

None

src/

|— hooks/

| |— useFetch.js

| |— useForm.js

PART 9: Clean Code Benefit

 Before:

- Repeated logic everywhere
- Hard to maintain

 After:

- Reusable logic
 - Clean components
 - Professional structure
-



KEY MINDSET

 Component = UI

 Hook = Logic



PRACTICE (VERY IMPORTANT)

 Q1:

Create `useFetch` hook

 **Q2:**

Add loading + error

 **Q3:**

Create `useForm` hook

 **Q4:**

Reuse hook in multiple components

MINI CHALLENGE

 Build:

☒ Custom Hook:

None

`useCounter`

 Features:

- increment
 - decrement
-

☒ **Example Output**

JavaScript

```
const { count, inc, dec } = useCounter();
```

Show more lines

INTERVIEW GOLD

👉 Question: What is a custom hook?

✅ Answer: 👉 *"A custom hook is a reusable JavaScript function that uses React hooks to share logic across components."*

FINAL UNDERSTANDING

👉 Think:

None

Hooks = reusable logic

Components = UI

NEXT STEP (BIG)

👉 Now we move to:

✅ PROJECT BUILDING PHASE

 Day 21+

We will build:

1. ☒ TODO App
2. ☒ Weather App (API)
3. ☒ Blog App
4. ☒ Full Dashboard